4주차 1일차: async/await로 기다리는 동안 화면 유지 — 30분

1. 학습 목표 — 2분

- 기다리는 작업이 화면을 멈추지 않도록 await를 사용합니다.
- Task를 반환하는 메서드를 AsyncRelayCommand에 연결합니다.

시작 상태는 3주차 완료본(day12)입니다. 오늘은 인터넷을 사용하지 않고 2초 대기만 흉내 냅니다.

2. 핵심 개념 설명 — 7분

Windows 화면은 보통 UI 스레드에서 입력과 그리기를 처리합니다. 이 스레드를 긴 대기로 붙잡으면 창 이동이나 입력도 멈춥니다.

Task는 진행 중인 작업을 나타냅니다. async 메서드 안에서 아직 끝나지 않은 Task를 await하면 그 메서드는 일단 제어를 돌려주고, 작업이 끝난 뒤 다음 줄을 이어서 실행합니다. await 자체가 새 스레드를 무조건 만드는 것은 아닙니다. C# 비동기 공식 안내

코드	역할
async Task RunAsync()	기다릴 수 있는 작업을 반환하는 메서드
await Task.Delay(2000)	UI 스레드를 붙잡지 않고 2초 대기
new AsyncRelayCommand(RunAsync, CanRun)	Task 반환 메서드를 버튼의 명령으로 제공

AsyncRelayCommand는 기본 설정에서 실행 중에는 CanExecute를 false로 알려 버튼 중복 실행을 막습니다. 이번에는 별도의 실행 중 플래그나 취소 버튼을 만들지 않습니다. 공식 AsyncRelayCommand 안내

.Result나 .Wait()로 강제로 기다리거나 Thread.Sleep으로 UI 스레드를 멈추지 않습니다. 오늘은 RunAsync가 시작할 때 입력값을 지역 변수에 복사하므로, 대기 중 입력창을 수정해도 결과는 클릭 당시 입력을 사용합니다.

3. 짧고 이해하기 쉬운 코드 예제 — 5분

```
public IAsyncRelayCommand RunCommand { get; }

public MainViewModel()
{
    RunCommand = new AsyncRelayCommand(RunAsync, CanRun);
}
```

처리 메서드는 다음과 같습니다.

```
    private async Task RunAsync()
    {
        string text = Input.Trim();
        Posts.Clear();
        Result = "처리 중입니다.";
        Status = "2초를 기다리는 동안에도 창을 움직일 수 있습니다.";

        await Task.Delay(2000);

        Posts.Add(new Post { Id = 1, Title = text });
        Result = "1건을 표시했습니다.";
        Status = "대기가 끝났습니다.";
    }
```

오늘 바뀌는 파일입니다. 본문은 핵심 부분만 보여 주므로 작성·교체에는 링크의 **전체 코드**를 사용합니다.

파일	전체 코드
MainWindow.xaml	day13의 MainWindow.xaml
MainViewModel.cs	day13의 MainViewModel.cs

4. Visual Studio에서 따라 하는 실습 — 13분

1. **2분:** 실행을 종료하고 MainViewModel.cs와 MainWindow.xaml을 오늘의 전체 코드로 교체합니다. 제목만 바뀌며 화면 배치와 Command 바인딩은 같습니다.
2. **4분:** Run이 RunAsync로 바뀌고, 반환형은 void 대신 Task가 된 것을 확인합니다. C# 파일에는 async void 메서드를 추가하지 않습니다.
3. **4분:** F5로 실행하고 입력 후 버튼을 누릅니다. 2초 동안 상태 문구가 보이고 버튼이 비활성화됩니다. 그동안 창을 움직이고 입력창의 텍스트를 바꿔 봅니다.
4. **3분:** 결과가 클릭 당시 텍스트로 표시되는지 확인합니다. 대기가 끝나면 버튼이 다시 활성화됩니다. 창을 닫습니다.

진도가 다르면 저장소 루트에서 powershell -ExecutionPolicy Bypass -File .₩sample₩Use-Stage.ps1 -Stage 12로 시작 상태를 적용합니다. 오늘의 완료본이 필요하면 Stage를 13로 지정합니다. 실행 중인 앱을 먼저 닫고 변경 파일을 다시 불러옵니다. 단계 적용·백업 안내

5. 이번 학습에서 반드시 이해해야 할 핵심 내용 요약 — 3분

- await는 비동기 작업이 끝날 때까지 기다린 뒤 다음 줄을 실행합니다.
- 기다리는 동안 UI 스레드가 다른 화면 작업을 처리할 수 있습니다.
- AsyncRelayCommand는 Task 반환 메서드를 연결하고 실행 중 버튼 상태를 관리합니다.
- 실무·면접 기초: async/await와 "새 스레드 생성"을 같은 의미로 설명하지 않습니다.

완료본: day13. 다음: 4주차 2일차.